

دليل إضافي

دليل إضافي: المشاركة بين الهواتف

ملف رزمة · Wi-Fi مباشر · مع القريبين (Nearby) · الأمان بالتوقيع والبصمة

الشاشات: شارك · أرسل على Wi-Fi · مع القريبين · استلم

وقت القراءة: 50 دقيقة

يسبقها: دليل تنزيل الدروس

تتبعها: —

المشروع: رِزمة · تطبيق تعلّم يعمل دون إنترنت

المنصة: Flutter · Dart · Drift · Riverpod · go_router

1. لماذا النقل من هاتف لهاتف؟

الإنترنت في غزة غالٍ ومقطوع، لكن الهواتف كثيرة والناس يتبادلون الملفات يدًا بيد (واتساب، بلوتوث، SHAREit، بطاقة ذاكرة). فيديو درس واحد نحو 30 ميغابايت: إن نزّله طالب واحد من الإنترنت يجب أن يكفي الصف كله.

الفكرة

هاتف واحد

ينزّل

من الإنترنت، ثم

ينشر

لهواتف كثيرة دون إنترنت. وكل هاتف يستلم يصير مصدرًا لغيره، كالسلسلة.

ملاحظة

دليل إضافي لميزة بُنيت بعد الخطة الأصلية، وقراراتها في `docs/adr/003-phone-to-phone.md` . تعتمد على نظام التنزيل (دليل تنزيل الدروس).

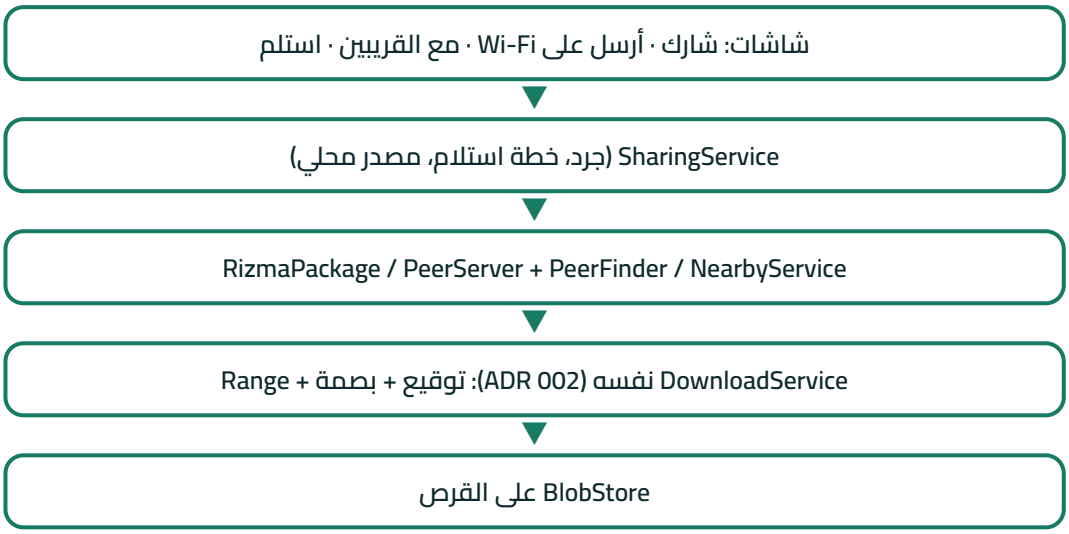
ثلاث طرق

الطريقة	كيف تعمل	تحتاج	الشاشة
ملف «رزمة» (`.rizma`)	ملف واحد يُرسل بأي وسيلة (واتساب، بلوتوث...) ثم «استلم من ملف»	لا شيء (تعمل دائمًا)	`share_screen.dart` و `receive_screen.dart`
Wi-Fi مباشر	خادم صغير على هاتف المرسل، والمستقبل يبحث عنه في الشبكة	شبكة Wi-Fi واحدة أو هوتسبوت	`send_wifi_screen.dart`
مع القريبين (Nearby)	الهواتف تجد بعضها بالبلوتوث وتنقل بـ Wi-Fi مباشر	خدمات Google + إذن «الأجهزة القريبة»؛ بلا إنترنت ولا شبكة	`nearby_send_screen.dart`

القرارات الأساسية (ADR 003)

القرار	لماذا
المصدر لا يُوثق به أبدًا: ما يصل من هاتف آخر يمرّ بنفس فحص التنزيل (توقيع الفهرس، رقمه لا ينزل، البصمة، الحجم حدٌّ أعلى)	هاتف غريب أو ملف معدّل لا يستطيع أن يُري الأطفال شيئًا لم ننشره
الهاتف يحفظ **توقيع الفهرس** مع الفهرس	ليعيد إرساله كما هو؛ لا يستطيع أي هاتف أن يوقّع
المصدر المحلي «مخزن» بنفس شكل R2	نفس كود التنزيل والإكمال والفحص، بلا مسار ثانٍ يُخطئ

القرار	لماذا
ملف تالف من هاتف آخر **لا يُحسب** على الملف	هاتف سيئ لا يوقف تنزيل الملف من الإنترنت لاحقًا
لا بيانات طلاب أبدًا: لا أسماء ولا نتائج ولا رموز	ما يُنقل محتوى عام فقط، مثل R2



2. الطريقة 1: ملف «رزمة» (.rizma)

الصيغة

صيغة .rizma — ADR 003 §3

```
RIZMA-PACK\n                          (11 بايت: الرأس)
طول الفهرس                           (4 بايت، big-endian)
الفهرس JSON: {"format":1,"entries":[{"path":"v1/grades/5/manifest.json","offset":0,"size":1234}, ...]}
البيانات بالترتيب
```

lib/features/sharing/rizma_package.dart — سطر 23

```dart
abstract final class RizmaPackage {
  static final magic = ascii.encode('RIZMA-PACK\n');
  static const format = 1;
  static const extension = 'rizma';

  static const maxIndexBytes = 4 * 1024 * 1024;
  static const maxEntries = 20000;

  static final _manifestPath = RegExp(
    r'^v1/grades/([1-9]|1[0-2])/manifest\.json(\.sig)?$',
  );
  static final _blobPath = RegExp(r'^v1/blobs/([0-9a-f]{64})$');
```

- **المسارات نفسها مسارات R2** (`v1/grades/<صف>/manifest.json` و `v1/blobs/<بصمة>`)، فيقرأ الهاتف الملف **كأنه مخزن** ويدعم Range ليكمل.

- ما يدخل: فهرس كل صف وتوقيعه، والكتالوج، وملفات كل درس مثبّت (صغيرة)، ثم ما اختاره المرسل من الفيديو والملفات. **الحجم ظاهر قبل الإرسال.**
- حدود دفاعية: الفهرس لا يتجاوز `maxIndexBytes` = 4MB، والمدخلات `maxEntries` = 20000، و `validPath` يقبل نمطين فقط (فهرس/توقيع، وبصمة 64 حرفًا).
- **الاستلام:** «استلم من ملف» ← يختار الملف ← يُفحص الفهرس ← يُضاف كل ما فيه وليس على الهاتف.

> **لماذا؟**
> `validPath` يرفض أي مسار غير مألوف (مثل `../../x`). هذا يمنع ملفًا خبيثًا من الكتابة خارج مخزن المحتوى.

الكتابة والقراءة

الصنف	وظيفته
`PackEntry`	مدخل في الملف: مسار + حجم + مصدر (بايتات أو ملف)
`RizmaPackage`	كتابة الملف (الرأس والفهرس والبيانات) وحساب حجمه
`PackageReader`	قراءة الفهرس وفحصه، وقراءة مدخل بـ Range
`PackageClient`	عميل HTTP وهمي يخدم طلبات GET من الملف، فيستعمل `DownloadService` كما هو

> **نصيحة**
> فكرة `PackageClient` عبقرية في بساطتها: بدل كتابة «مسار استلام» جديد، نجعل الملف يتظاهر بأنه خادم. فيعمل كل كود التنزيل والفحص بلا تعديل.

3. الطريقة 2: Wi-Fi مباشر

المرسل

- «أرسل على Wi-Fi» يفتح **خادمًا صغيرًا على المنفذ 47123** (`Peer.port`) **ما دامت الشاشة مفتوحة فقط**، ويعرض **رمزًا من 4 أرقام** (`1000 + Random.secure().nextInt(9000)`).
- يقدّم (للقراءة فقط): `rizma/hello` (الرمز) و `rizma/inventory.json` (الصفوف وبصمات ما عنده) والفهارس وتوقيعاتها والملفات بنفس مسارات R2.
- **يبقي الشاشة مفتوحة** (`WifiBinding.keepScreenOn` عبر قناة أصلية في `MainActivity.kt`) كي لا ينام الهاتف ويتوقف الخادم.

lib/features/sharing/peer.dart سطر 174

```
static Future<PeerServer> start(
  ShareInventory inv,
  BlobStore store, {
  int port = Peer.port,
  String? code,
  InternetAddress? address,
}) async {
  final server = await HttpServer.bind(
    address ?? InternetAddress.anyIPv4,
    port,
  );
  final peer = PeerServer._(
    server,
    code ?? (1000 + Random.secure().nextInt(9000)).toString(),
    inv,
    store,
  );
  server.listen(peer._handle, onError: (_) {});
  return peer;
}
```

المستقبِل

- «ابحث عن هاتف قريب» ← `PeerFinder` يجرّب **كل عناوين شبكته (/24)** على المنفذ نفسه، ويستجيب أي هاتف رِزمة بـ `rizma/hello` (يتأكد أن `app == 'rizma'`).
- يعرض الهواتف برموزها، ويمكن كتابة العنوان يدويًا إن لم يُعثر عليه.
- يختار المستقبل ما يريد (الدروس، الفيديو، الملفات بأحجامها) ويكمل التنزيل من حيث توقف إن انقطع.
- فشل البحث له أسباب واضحة (`PeerFailure`): ليس رِزمة، لا يستجيب، … وتعرضها الواجهة.

lib/features/sharing/peer.dart سطر 324

```
Future<PeerInfo?> hello(String host) async {
  final base = Uri(scheme: 'http', host: host, port: port, path: '/');
  try {
    final res = await _client
        .get(base.resolve('rizma/hello'))
        .timeout(timeout);
    Object? json;
    try {
      json = res.statusCode == 200 ? jsonDecode(res.body) : null;
    } on FormatException {
      json = null;
    }
    if (json is! Map || json['app'] != 'rizma' || json['code'] is! String) {
      lastFailure = PeerFailure.notRizma;
      return null;
    }
    lastFailure = null;
    return PeerInfo(baseUrl: base, code: json['code'] as String);
  } catch (e) {
    lastFailure = PeerFailure.of(e);
```

لماذا HTTP عادي؟

داخل الشبكة المحلية (ADR 003 §4): أندرويد 9+ يمنع HTTP العادي افتراضيًا، وعناوين الشبكة المحلية تتغيّر (192.168.x.x) فلا تُحصر بالاسم. لذلك يُسمح به في `network_security_config.xml` للتطبيق (قرار صاحب المشروع، 30 أكتوبر 2026).

> **انتبه**
>
> **الأمان لا يأتي من الطريق بل من التوقيع والبصمة.**
>
> كل اتصالنا بالإنترنت HTTPS، والمحتوى موقّع ومفحوص أيًّا كان مصدره. فحتى لو اعترض أحد الشبكة المحلية لا يستطيع أن يُري الطالب محتوى لم يوقّعه فريق المحتوى. (وتوسيع السماح بـ HTTP ليشمل التطبيق كله قرار تاريخي ويمكن تضييقه لاحقًا.)

4. الطريقة 3: «مع القريبين» (Nearby)

Wi-Fi يحتاج شبكة أو هوتسبوت يشغّله الطالب، وهذه خطوات كثيرة لطفل. لذلك أُضيفت طريقة ثالثة: **Nearby Connections من Google** (حزمة `nearby_connections`): الهواتف تجد بعضها بالبلوتوث وتنقل الملفات بـ Wi-Fi مباشر بينها. **بلا إنترنت ولا راوتر ولا هوتسبوت**، والمرسل «نجمة» يتصل به عدة هواتف معًا.

القناة والجلسة

lib/features/sharing/nearby_protocol.dart — سطر 11

```
abstract class NearbyChannel {
  Future<void> sendMessage(Map<String, Object?> message);

  Future<int> sendFile(File file);

  Stream<Map<String, dynamic>> get messages;

  Stream<(int, File)> get files;

  Stream<(int, int, int)> get progress;

  Future<void> close();
}
```

- Nearby **قناة رسائل وملفات** لا شبكة IP. فالمستقبل **يحوّل طلبات GET نفسها إلى رسائل** (`NearbyClient extends http.BaseClient`)، والمرسل يجيب بملف (`NearbyShareSession`).
- ومن ثمّ كل ما بعدها هو **كود التنزيل والفحص نفسه** عبر مصدر محلي (`localSource`) — التالف يُرمى ولا يُحسب.
- المرسل يسمح فقط بمسارين: `v1/blobs/<64 hex>` و `v1/grades/<n>/manifest.json[.sig]` (تعبيران نمطيان ثابتان)، فلا يقرأ المستقبل أي ملف آخر من هاتفه.

lib/features/sharing/nearby_protocol.dart — سطر 40

```
static final _blobPath = RegExp(r'^v1/blobs/([0-9a-f]{64})$');
static final _manifestPath = RegExp(
  r'^v1/grades/(\d{1,2})/manifest\.json(\.sig)?$',
);
```

الخدمة والمشاكل

الصنف	وظيفته
`NearbyService`	يبدأ الإعلان والاكتشاف والاتصال
`NearbyPeer`	هاتف قريب (معرّف واسم)
`NearbyProblem`	`permission` (إذن الأجهزة القريبة)، `locationOff` (الموقع على أندرويد قبل 13)، `unsupported` (بلا خدمات Google)
`NearbyProblemView`	تعرض المشكلة وتوجّه المستخدم للحل

> **ملاحظة**
> هاتف بلا خدمات Google يُوجَّه إلى الهوتسبوت (القسم 3). وطريقة الملف (القسم 2) تعمل دائمًا.

«انشر الدروس» بعد الاستلام

بعد كل استلام ناجح يظهر زر يفتح «شارك» مباشرة، فيصير كل هاتف استلم مصدرًا لغيره.

5. SharingService وشاشة الاستلام

الجرد وخطة الاستلام

lib/features/sharing/sharing_service.dart — سطر 35

```
class ReceivePlan {
  const ReceivePlan({required this.items, required this.accepted});

  final List<DownloadItem> items;

  final bool accepted;

  int get size => items.fold(0, (s, i) => s + i.size - i.received);
}
```

- `ShareInventory` (من `DownloadService.shareInventory`): ما عندي من صفوف وفهارس وتوقيعات وبصمات (core) والوسائط (`media`: فيديو ومرفقات).
- `packEntries(inventory, mediaIds)`: ما يدخل ملف الرزمة (الفهارس والتوقيعات + ملفات core + ما اختاره المرسل من الوسائط).
- `ReceivePlan`: ما سيُستلم (العناصر) والحجم المتبقي (`size - received`) و`accepted` (هل قُبل الفهرس).
- `localDownloads(source)`: ينشئ `DownloadService` يقرأ من مصدر محلي بدل R2: **الكود نفسه، المصدر مختلف**.

ReceiveScreen: مراحل الاستلام

lib/features/sharing/receive_screen.dart سطر 28

```
enum _Stage { idle, nearby, searching, opening, choosing, receiving, done }
```

المرحلة	ما يحدث
`idle`	البداية: اختيار الطريقة (قريبين، Wi-Fi، ملف)
`searching` / `nearby`	البحث عن هواتف قريبة
`opening`	فتح اتصال أو ملف وفحص فهرسه
`choosing`	اختيار ما يُستلم (الدروس، الفيديو، الملفات بأحجامها)
`receiving`	تنزيل الملفات مع الإكمال عند الانقطاع
`done`	اكتمل: زر «انشر الدروس»

الشاشة كبيرة (692 سطرًا) لأن كل مرحلة بودجت خاص. `_Busy` و `_Searching` و `_Start` أمثلة.

6. الإعدادات الأصلية (Android)

الملف	ما فيه
`android/app/src/main/AndroidManifest.xml`	أذونات: INTERNET ACCESS_NETWORK_STATEg ACCESS_WIFI_STATEg (وإذونات Nearby تضيفها الحزمة)
`android/app/src/main/res/xml/network_security_config.xml`	السماح بـ HTTP للشبكة المحلية (cleartextTrafficPermitted)
`android/app/src/main/kotlin/com/rizma/rizma/MainActivity.kt`	قناة أصلية: `keepScreenOn` وربط الشبكة (`WifiBinding`)

> **انتبه**
>
> **iOS:**
>
> لا يوجد `NSLocalNetworkUsageDescription` في `ios/Runner/Info.plist` ؛ فنقل Wi-Fi المحلي لن يعمل على iOS ما لم يُضَف. المشروع حاليًا يستهدف أندرويد، والإضافة مطلوبة إن دُعم iOS.

خارج هذا القرار

- فتح ملف `.rizma` مباشرة من واتساب (Open with). اليوم: يُحفظ ثم يُختار من التطبيق.
- Wi-Fi Direct بلا شبكة ولا هوتسبوت.
- نقل نتائج الطالب أو ملفه بين هاتفين: يمرّ بالمزامنة (دليل المزامنة).

7. الاختبارات وطريقة التجربة

الملف	ما يغطيه
`test/features/sharing_test.dart` (9)	ملف الرزمة: كتابة وقراءة، ملف معدَّل، فهرس أقدم، توقيع خاطئ، ملف أكبر من حجمه، انقطاع في منتصف النقل
`test/features/peer_test.dart` (16)	الخادم المحلي والبحث عن الهواتف (`PeerServer` و `PeerFinder`) وحالات الفشل
`test/features/nearby_test.dart` (4)	بروتوكول Nearby: الطلبات والمسارات المسموحة والملفات
`test/features/sharing_ui_test.dart` (14)	شاشات «شارك» و«استلم» بمراحلها

> **نصيحة**
> شغّل: `flutter test test/features/sharing_test.dart test/features/peer_test.dart test/features/nearby_test.dart`

جرّبها بنفسك

1. جهاز أول: «حسابي ← إدارة التنزيلات ← شارك» ← اختر محتوى وأنشئ ملف «رزمة» وأرسله لجهاز ثانٍ.
2. جهاز ثانٍ: «استلم من ملف» ← تلاحظ فحص الفهرس ثم قائمة ما سيُضاف.
3. جرّب Wi-Fi: على الأول «أرسل على Wi-Fi» (يظهر رمز من 4 أرقام)، وعلى الثاني «ابحث عن هاتف قريب».
4. اقرأ `peer_test.dart` لترى كيف تُحاكى الشبكة دون هاتف حقيقي.

8. تدرّب وتحقّق من فهمك

قرارات تصميم وأسبابها

القرار	السبب
ملف الرزمة بمسارات R2	كود التنزيل نفسه يقرأه كمخزن.
القراءة للقراءة فقط وبمسارين مسموحين	لا يستطيع المستقبل قراءة ملفات أخرى من هاتف المرسل.
الخادم المحلي ما دامت الشاشة مفتوحة	لا يبقى خادم يعمل في الخلفية دون علم الطالب.
HTTP للشبكة المحلية فقط عمليًا	عناوينها تتغيّر؛ الأمان من التوقيع والبصمة.
Nearby يحوّل GET إلى رسائل	إعادة استعمال كامل منطق التنزيل.
ملف تالف من هاتف آخر لا يُحسب	هاتف سيئ لا يمنع التنزيل من الإنترنت لاحقًا.

أسئلة

س1. هل نثق بملف رزمة يصلنا من صديق؟ ماذا نفعل به؟

س2. لماذا يعرض المرسل رمزًا من 4 أرقام؟ وكم احتمالًا له؟

س3. لماذا لا يعمل خادم Wi-Fi في الخلفية؟

س4. ما الذي يمنع مستقبلًا خبيثًا من قراءة ملفات هاتف المرسل؟

س5. هاتف بلا خدمات Google. أي الطرق الثلاث تعمل؟

تمارين

1. اكتب اختبارًا أن `RizmaPackage.validPath` يرفض مسارًا فيه «..».
2. اكتب اختبارًا أن رقم `PeerServer` الافتراضي دائمًا بين 1000 و9999.
3. اقرأ ADR 003 وحدّد ما الذي يحتاج تغييرًا لدعم iOS.

الإجابات

ج1. لا نثق بأي مصدر. الملف يمرّ بنفس فحص التنزيل: توقيع الفهرس، ورقم الفهرس لا ينزل، والبصمة، وحد الحجم.

ج2. ليعرف المستقبل أي هاتف في الغرفة هو المقصود (عدة هواتف قد تُرسل). وله 9000 احتمال (1000–9999). ليس أمانًا بل تمييزًا.

ج3. حتى لا يبقى خادم مفتوح بلا علم الطالب، ولأن الهاتف قد ينام فيتوقف.

ج4. المرسل يسمح فقط بمسارين بتعبيرين نمطيين (فهرس/توقيع، وبصمة 64 حرفًا)، فكل مسار آخر مرفوض.

ج5. ملف رزمة (دائمًا) Wi-Fig/هوتسبوت. أما Nearby فيحتاج خدمات Google.

مصطلحات

المصطلح	المعنى
.rizma	ملف رزمة: دروس في ملف واحد
PeerServer	الخادم المحلي على هاتف المرسل
PeerFinder	يبحث عن هواتف رِزمة في الشبكة
Nearby Connections	نقل بين الهواتف القريبة من Google
Hotspot	هاتف يبثّ شبكة Wi-Fi

المصطلح	المعنى
Inventory	جرد ما عند المرسل
Local source	مصدر ملفات محلي بدل R2

ما بعد هذا الدليل

هذا آخر الأدلة. للتنسيق بين الأعضاء انظر `docs/study_guides/00_فهرس_الأدلة.pdf` .